Page No.:
Date:

8] Write a c++ program to perform arithmetic operations

→

```
#include <iostream.h>
int main()
{
double num1, num2;
char op;
Cout << " Enter 1st number : ";
Cin >> num1;
Cout << "Enter operator (+, - , * , / )";
 Cin >> op;
Cout << "Enter 2nd number : ";
 Cin >> num2;
switch (op)
{
 case '+' ;
Cout << " Result: " << num1 + num2 << endl;
 break ;
 case '-' ;
Cout << " Result: " << num1 - num2 << endl;
 break;
case '*' ;
Cout << "Result: " << num1 * num2 << endl;
break;
case '/' ;
[Cout << " Result: " << num1 / num2 << endl;
break;]
Cout << " Result: " << num1 / num2 << endl;
 break;
```

```
if (num 2 != 0)
cout << " Result : "<<num 1 / num 2 << en
else
cout << " Error : Division ! "<< end l ;
break;
default :
cout << " Invalid operator" << end l;
}
return 0;
}
```

Output

Enter 1st number : 12
enter operator (+, -, *, /) *
Enter 2nd number : 12
Result : 144

Page No.:
Date:

8] Write a C++ program to check number is even or odd.

→
```
#include <iostream>
using namespace std;
int main()
{
int num;
Cout << "Enter an integer:";
Cin >> num;
if (num % 2 == 0)
Cout << num << " is even". << endl;
else
Cout << num << " is odd". << endl;
return 0;
}
```

Output

```
Enter an integer: 6
6 is even
```

Q] Write a c++ program to print numbers from 1 to 10

→
```
#include <iostream>
using namespace std;
int main()
{
cout << "Numbers from 1 to 10: " << endl;
for (int i=1; i<=10; i++)
{
cout << i << " ";
}
cout << endl;
return 0;
}
```

Output

Numbers from 1 to 10

1 2 3 4 5 6 7 8 9 10

Page No.:
Date:

Q] Write a C++ program to print numbers from 10 to 1.

→
```
# include <iostream>
using namespace std;
int main ()
{
int i = 10;
cout << " Numbers from 10 to 1 ; " << endl;
while (i >= 1)
{
cout << i << " ";
i--;
}
cout << endl;
return 0;
}
```

Output

Numbers from 10 to 1

10 9 8 7 6 5 4 3 2 1

Q]

```
    *
  * *
* * *
```

→

```
#include <iostream>
using namespace std;
int main()
{
int rows = 3;
for (int i = 1; i <= rows; i++)
{
for (int j = 1; j <= rows-i; j++)
{
cout << " ";
}
for (int k = 1; k <= i; k++){
{                          cout << "*";
                           }
cout << endl;
}
return 0;
}
```

Output

```
    *
  * *
* * *
```

Page No.:
Date:

Q] 1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

→
```
# include <iostream>
using numspace std;
int main ()
{
int rows = 5;
for (int i = 1; i <= rows; i++)
 {
  for (int j=1 ; j <= i ; j++)
    {
   cout << i << " ";
  }
  cout << end l;
}
return 0;
}
```

Output

1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

8]
```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

→
```
# include <iostream>
using namespace std;
int main()
{
 int rows = 5;
 for (int i = 1 ; i <= rows ; i++)
 {
  for (int j = 1 ; j <= i ; j++)
  {
   cout << j << " ";
  }
  cout << endl;
 }
 return 0;
}
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Page No.:
Date:

Q] Write a C++ program to add two numbers.

→
```
# include <iostream>
void main ()
{
int a, b, c;
Cout << " enter value of a & b";
Cin >> a >> b;
 c = a+b;
Cout << " Addition: " << c;
}
```

Output

Enter value of a & b : 2 4
Addition : 6

Qn
8/7/25

Experiment 1

Write a c++ program to declare a class stud
having data members as Roll_no & name
Accept and display data for single stude

```
#include <iostream>
using namespace std;
Class Student
{
  int Roll_no;
  string name;
  Public;
  void accept()
  { cout << " Enter name & roll no :";
    cin >> name >> Roll_no;
  }
  void display ()
  {
   cout << " Name : " << name;
   cout << " Roll no : " << Roll_no;
  };

  int main()
  {
   Student S1;

   S1 . accept();
   S1 . display(); }
   return 0;
  }
```

Page No.:
Date:

Output

Enter name and Roll no: Spruha
25

Name: Spruha
Roll no: 25

Write a C++ code to create a class book having data members as book_name, b_price, b_pages. Accept the data for two books and display the name of book having greater price.

```
#include <iostream>
using namespace std;
Class book
{ Public :
  string b_name ;
  int b_pages ;
  int b_price ;
  Public :
  void accept ()
{ Cout << " Enter book name, book price and
          book pages : " ;
  Cin >> b_name >> b_price >> b_pages ;
}
Void display ()
{ Cout << " Book name : " << b_name ;
  Cout << " Book price : " << b_price ;
  Cout << " Book pages : " << b_pages ;
};

int main ()
{ Book B1, B2 ;

  B1 . Accept () ;
  B2. accept () ;
```

```
if (B1.price> B2.price)
 {
   B1. display ();
 }
 else
 {
   B2. display ();
 }
 return 0;
}
```

Output

enter book name, no. of ~~prig~~ pages and price :
Alchemist
45
500

enter bookname, no. of pages and price :
Youngminds
235
700

bookname : youngminds
bookprice : 700
boo no. of pages : 235

Write a C++ program to declare class time accept time in HH:MM:SS format, convert into seconds & display.

```
# include <iostream>
using namespace std;

class timesec
{
int h,m,s;
char c;
public:
void accept()
{
cout << "Enter time in HH:MM:SS format:"
cin >> h >> c >> m >> c >> s;
}
void display()
{
int totalseconds = 3600* h + m * 60 + s;
cout << "Time in seconds:" << totalseconds << en
}
};

int main()
{
    timesec T;
    T.accept();
    T.display();
}
```

```
    return 0;
}
```

Output

```
Enter time in HH:MM:SS format : 23:22:23
Time in seconds : 84143
```

Q1
15/7

Page No.:
Date:

Experiment 2

WAP to declare a class 'city' having data members as name and population. Accept this data for 5 cities having highest popula

```
#include <iostream>
using namespace std;
class city
{public:
 string name;
 int population;
 public:
 void accept()
{
cout<<" enter the name of city :"<<endl;
cin>> name ;
cout<<" enter population :"<< endl;
cin>> population;
}
void display ()
{
cout<<"Name :"<<name<<endl;
cout<<" Population :"<<population<<endl;
}
};
city c[5];
int main ()
{
int i;
 for (i=0; i<5; i++)
 {
  c[i].accept()
```

```
}
int max = 0;
for (i=0; i<5; i++)
{
  if ( c[i].population > c[max].population)
    {
      max = i;
    }
}
cout << "The city with maximum population:" <<
endl;
c[max].display ();
return 0;
}
```

Output

enter name of city:
city 1: Pune
Population: 23457
city 2: Katraj
Population: 23574
city 3: Amritsar
Population: 34780
city 4: Mahabaleshwar
Population: 23008
city 6: Vai
Population: 21007

The city with maximum population: 34780

WAP to declare a class 'Account' having data members as Account no. and balance. Accept this data for 10 accounts and give interest of 10% where balance is equal or greater than 5000 and display.

```
#include <iostream>
using namespace std;
class account
{
  int accno;
  int balance;
public:
  void accept()
{
cout << " Enter account number: " << endl;
cin >> accno;
cout << " Enter balance : " << endl;
cin >> balance;
}
void display()
{
cout << " Account number: " << accno << " Balance: "
                                balance << endl;
}

void addinterest()
{
 if (balance >= 5000)
  {
```

```
        int interest = balance * 0.10;
        balance += interest;
        cout << "Interest of 10% added." << balance <<
                                                endl;
    }
    else
    {
    cout << "Balance is less than 5000, no interest
            is added." << endl;
    }
}

int getBalance()
  {
    return balance;
  }
};

int main()
{
  account accounts[10];
    int numAccounts;

cout << "Enter number of accounts (max 10): ";
cin >> numAccounts;

  int
for(i=0; i < numAccounts; i++)
{
cout<" \n Account " << (i+1) << ": " << endl;
accounts[i].accept();
}
```

```
cout << "\n Account Display with 10% interest" << en
for(i=0 int i=0 ; i < numAccounts ; i++)
 {
 cout << "\n Account" << (i+1) << " : " << endl;
 account[i].display();
 accounts[i].addInterest();
}
 return 0;
}
```

Output

Page No.:
Date:

WAP to declare a class 'Staff' having data members as book_title, author, name & price. Accept & display the info. for 1 object using pointer of that object.

```
#include <iostream>
using namespace std;
class staff
{
 string name;
 string post;
public:
 void accept()
 {
  cout << "Enter name:";
  cin >> name;
  cout << "Enter post:";
  cin >> post;
 }
 void display()
 {
 cout << "Name:" << name << "Post:" << post << endl;
 }
 string getPost()
 {
  return post;
 }
 string getName()
 {
  return name;
 }};
```

```
int main()
{
 Staff S[5];
 bool foundHOD = false;

 for(int i=0; i<5; i++)
 {
 cout<<" Staff member "<<(i+1)<<" :"<<endl;
  S[i].accept();
 }
 for(int i=0; i<5; i++)
 {
  if(S[i].getPost() == "HOD")
  {
  cout<<"HOD is :"<<S[i].accept();<<endl;
  foundHOD = True;
  break;
  }
 }
 if (!foundHOD)
  cout<<" HOD not found. "<<endl;
 return 0;
}
```

Output

```
Name: Krishana
Post: HOD
Name: Sara
Post: Staff
```

Page No.:
Date:

Name : Aishni
Post: Staff
Name: Neel
Post: Administrator
Name: Swati
Post: Head

HOD is : Krishna

29/7/25

Experiment 3

1. WAP to declare a class 'book' having data members as book-title, author-name and price. Accept and display the information for one object using a pointer to that object.

```
#include <iostream>
#include <string>
using namespace std;
class books
{
  string name;
  string author;
  int price;
  public:
  void accept(){
  cout<<" Book Name :"<<endl;
  cin >> name;
  cout <<" Name of Author :"<< endl;
  cin >> author;
  cout <<"Book price :"<<endl;
  cin >> price;
  }
  void display(){
cout <<"Book Name :"<<name<<" Name of Author:"<<
  author <<" Book price :" << price;
  }
};
```

Page No.:
Date:

```
int main(){
 books B;
 books *p;
 p = &B;
 p -> accept();
 p -> display();
 return 0;
}
```

WAP to declare a class 'student' having data mem
as roll-no and percentage. Using 'this' pointe
invoke member functions to accept and displa
this data for one object of the class.

```
#include <iostream>
# using namespace std;

class student
{
  int roll_no;
  string name;
  float marks[5];
  float total_marks;
  float percentage;
  public:
  void accept(){
  cout<<" Enter Roll no: "<<endl;
   cin>> this -> roll_no;
   cout<<" Name :"<<endl;
   cin >> this -> name;
   cout<<"Enter marks for 5 subjects:"<<end
    total_marks = 0;
  for(int i=0; i<5; i++){
    cout<<"Subject "<<(i+1)<<":";
    cin >> marks[i];
    total_marks += marks[i];
  }
    calculate Percentage();
}
```

```
void calculatePercentage(){
    percentage = (total_marks/500) * 100;
}
void display(){
    cout<<"Roll No: "<<roll_no<<endl;
    cout<<"Name: "<<name<<endl;
    cout<<"Total marks: "<<total_marks<<"/500"<<
                                        endl;
    cout<<"Percentage: "<<percentage<<"%"<<endl;
    }
};

int main(){
    student S1;
    S1.accept();
    S1.display();
    return 0;
}
```

WAP to demonstrate the use of nested class as Student and Marks:

```
#include <iostream>
using namespace std;

class Student
{
  int rollNo;
  String name;
public:
  void getStudentData() {
  cout << "Enter Roll Number:";
  cin >> rollNo;
  cout << " Enter Name:";
  getline (cin, name);
    cin >> name;
  }

  void displayStudentData() {
   cout << " Student Details:";
   cout << "\n Roll Number:" << rollNo;
   cout << " \n Name: " << name << endl;
  }
  class Marks {
    int m1, m2;
  public:
    void getmarks() {
    cout << " Enter marks for 2 subjects:";
    cin >> m1 >> m2;
    }
```

```
void displayMarks() {
    cout << "\n Marks Obtained: ";
    cout << "\n Subject 1: " << m1;
    cout << "\n Subject 2: " << m2;
    int total = m1 + m2;
    float percentage = total / 2.0;
    cout << "\n Total Marks: " << total;
    cout << "\n Percentage: " << percentage << "%" << endl;
  }
};
};

int main() {
    Student S;
    s.getStudentData();
    s.displayStudentData();

    Student::marks m;
    m.getmarks();
    m.displayMarks();

    return 0;
}
```

5/8/25

Experiment 6

1) WAP to swap two numbers from same class using objects as function argument. Write swap function as member function.

→

```
#include <iostream>
using namespace std;

class Number {
    int a,b;
public:
    void Data (int x, int y) {
        a = x;
        b = y;
    }
    void display () {
    cout << " a = " << a << " b = " << b << endl;
    }

    void swap ( Number & obj)
     int temp;

     temp = a;
     a = obj.a;
     obj.a = temp;

     temp = b;
     b = obj.b;
     obj.b = temp;
    }
};
```

Page No.:
Date:

```
int main(){
    Number obj1, obj2;
    obj1.Data(10,20);
    obj2.Data(30,40);

    cout<<" Obj 1 : ";
    obj1.display();
    cout<<" Obj 2: ";
    obj2.display();

    obj1.swap(obj2);

    cout<<"\nAfter Swap : "<<endl;
    cout<<" Obj 1 :";
    obj1.display();
    cout<<" Obj 2 :";
    obj2.display();
    return 0;
}
```

Output

Obj 1 : a = 10, b = 20
Obj 2 : a = 30, b = 40

After Swap:
Obj 1 : a = 30, b = 40
Obj 2 : a = 10, b = 20

2) WAP to swap two numbers from same class using Friend Function

```
#include <iostream>
using namespace std;

class Number {
    int value;
public:
    void value(int v) {
        value = v;
    }
    void display() {
        cout << "value: " << value << endl;
    }
    friend void swapValue(Number &N1, Number &N2);
};

void swapValues(Number &N1, Number &N2) {
    int temp = N1.value;
    N1.value = N2.value;
    N2.value = temp;
}

int main() {
    Number A, B;
    A.value(10);
    B.value(20);
```

```
A.display();
B.display();

swapvalues(A,B);

cout<<"New values:"<<endl;
   A.display();
   B.display();
return 0;
}
```

Output

A = 10
B = 20
New values:
A = 20
B = 10

3) WAP to swap two numbers from different class using friend function.

→

```
#include <iostream>
using namespace std;

class ClassB;
class ClassA {
    int numA;
public:
    void value(int a) {
        numA = a;
    }
    void display() {
    cout<<" ClassA Value = " << numA << endl;
    }
    friend void swapvalues(ClassA &, ClassB &);
};

class ClassB {
    int numB;
public:
    void value(int b) {
        numB = b;
    }
    void display() {
    cout<<" ClassB Value = " << numB << endl;
    }
friend void swapvalues(ClassA &, ClassB &);
};
```

Page No.:
Date:

```
void swapValues (ClassA &a, ClassB &b) {
    int temp = a.numA;
    a.numA = b.numB;
    b.numB = temp;
}

int main() {
    ClassA A;
    ClassB B;

    A.value(20);
    B.value(30);
    cout<<"Values: "<<endl;
    A.display();
    B.display();
    swapValues(A, B);
    cout<<"Swapped Values: "<<endl;
    A.display();
    B.display();
    return 0;
}
```

4) WAP to create two classes Result1 & Result2 which stores the marks. Read values of both & classes object & compute the average of two results.

→

```
#include <iostream>
using namespace std;
class Result2;
class Result1{
  float marks1;
public:
  void readmarks(){
  cout<<"Enter marks for Result1:";
    cin >> marks1;
  }
float
friend float computeAverage(Result1 &r1, Result2 &r
};

class Result2{
  float marks2;
public:
  void readmarks(){
  cout<<"Enter marks for Result2:";
  cin >> marks2;
  }
friend float computeAverage(Result1 &r1, Result2 &r2);
};
```

```
float computeAverage(Result1 r1, Result2 r2) {
    return (r1.marks1 + r2.marks2) / 2;
}

int main() {
    Result1 A1;
    Result2 A2;

    A1.readmarks();
    A2.readmarks();

    float avag = computeAverage(A1, A2);
    cout << "\n Average of two results: " << avag << endl;
    return 0;
}
```

Output

```
Enter marks for Result1: 10
Enter marks for Result2: 20

Average of two results: 15
```

Page No.:
Date:

5) WAP to find the greatest number among two numbers from different classes using friend function.

→

```
#include <iostream>
using namespace std;
class ClassB;
class ClassA {
  int numA;
public:
  void Accept(int a) {
  ~~cout<<" Class A Number: "<<numA<<endl;~~
    numA = a;
  }
  void display() {
  cout<<" ClassA Number: "<<numA<<endl;
  }
  friend void findgreatest(ClassA, ClassB);
};

class ClassB {
  int numB;
public:
  void Accept(int b) {
    numB = b;
  }
  void display() {
  cout<<" ClassB Number: "<<numB<<endl;
  }
  friend void findgreatest(ClassA, ClassB);
};
```

```
void findgreatest (ClassA a, classB b){
  if (a.numA > b.numB){
  cout<<" Greatest number is from class A."<< a.numA
                                          <<endl;
  } else if ( b.numB > a.numA){
  cout<<" Greatest number is from class B."<<b.numB
                                          <<endl;
  } else {
    cout <<" Both are equal."<<endl;
  }
}

int main(){
  ClassA A;
  classB B;

    A.Accept(35);
    B.Accept(40);
    A.display();
    B.display();

    findgreatest(A, B);
    return 0;
}
```

Output

Class A Number: 35

Class B Number: 40

Greatest number is from ClassB: 42

6) Create two classes with seperate obj. Write a friend function sum() that can access data from both the classes.

→

```
#include <iostream>
using namespace std;
class ClassB;
class ClassA {
    int numA;
public:
    void accept(int a) {
        numA = a;
    }
    void display() {
    cout << "Enter Number: " << endl;
    cin >> numA;
    }
friend int sum(ClassA &, ClassB &);
};

class ClassB {
    int numB;
public:
    void accept(int b) {
        numB = b;
    }
    void display() {
    cout << "Enter number: " << endl;
    cin >> numB;
    }
friend int sum(ClassA &, ClassB &);
```

```
};
int void intsum ( class A &, class B &){
    return a.numA + b.numB;
}
int main(){
    class A  A(78);
    class B  B(67);

    intsum ( A &, B)

    cout << " Sum = " << sum (A, B) << endl;
    return 0;
}
```

Output

Sum = 145

7) WAP to swap numbers of same class using friend function.

→

```
#include <iostream>
using namespace std;
class Number {
    int value;
private:
    int value;
public:
```

```
Number (int val) : value(val){}
void show(){
  cout<<" Value:"<<value<<endl;
}
friend void swapValues(Number &, Number &);
};

void swapValues(Number &n1, Number &n2){
  int temp = n1.value;
  n1.value = n2.value;
  n2.value = temp;
}

int main(){
  Number num1(56);
  Number num2(44);

cout<<"Before swap:"<<endl;
  num1.show();
  num2.show();

  swapValues(num1,num2);
  cout<<" Swap:"<<endl;
  num1.show();
  num2.show();
  return 0;
}
```

~~Output~~

8) Define two classes Box & Cube, each having a private volume. Write a friend function find greatest (Box, Cube) that determines which object has larger volume.

→
```
#include <iostream>
using namespace std;
class Cube;
class Box {
private:
    int volume;
public:
    Box(int v) : volume(v) { }
    friend void findGreater ( Box & b1, Cube & c1);
};

class Cube {
private:
    int volume;
public:
    Cube (int v) : volume(v) { }
    friend void findGreater ( Box & b1, Cube & c1);
};

void findGreater ( Box b1, Cube c1){
if (b.volume > c.volume){
cout << " Volume of Box is greater." << endl;
} else if( c.volume > b.volume){
cout << " Volume of Cube is greater." << endl;
} else {
cout << " Equal volumes." << endl;
}
```

```
int main() {
    Box box1(5.0);
    Cube cube1(5.0);

    findGreater(box1, cube1);
    return 0;
}
```

9) Create a class for complex with reals imaginary part as private members. Use friend function show their sum.

```
#include <iostream>
using namespace std;

class Complex {
private:
    int real;
    int imag;
public:
Complex(int r=0, int i=0) { real=r; imag=i; }

void show() {
    cout << real << " + i" << imag << ";" << endl;
}
friend complex add(complex, complex);
};
```

```
Complex add(complex c1, complex c2) {
    complex result;
    result.real = c1.real + c2.real;
    result.imag = c1.imag + c2.imag;
    return result;
}
int main() {
    Complex a(2,3);
    Complex b(4,5);
    Complex c = add(a,b);
    cout << " sum of complex numbers: ";
    c.show();
    return 0;
}
```

10) Create a class student with private data members name & three subject marks. Write a friend function to calculate average.

```
#include <iostream>
using namespace std;
class student {
private:
    string name;
    int marks1, marks2;
    int marks3;
public:
    student(string n, int m1, int m2, int m3) {
    name = n;
    marks1 = m1;
```

```
    marks2 = m2;
    marks3 = m3;
  }
  friend void calculateAverage(Student);
};
void calculateAverage(Student s) {
    float avg = (s.marks1 + s.marks2 + s.marks3)
  cout << "Student Name: " << s.name << endl;
  cout << "Average marks: " << avg << endl;
}
  int main(){
  student s1("Ani", 85, 90, 95);
  calculateAverage(s1);
  return 0;
  }
```

11) Create three classes: Alpha, Beta & Gamma each with private members. Write a friend function that can access all & print their sum.

```
#include <iostream>
using namespace std;
class Beta;
class Gamma;
class Alpha {
private:
  int a;
public:
  Alpha(int val) {
    a = val;
  }
```

```
friend void totalSum( Alpha, Beta, Gamma );
};
class Beta {
private:
  int b;
public:
  Beta( int val) {
     b = val;
  }
  friend void totalSum( Alpha, Beta, Gamma );
};
class Gamma {
private:
  int c;
public:
  Gamma (int val) {
     c = val;
  }
  friend void totalSum (Alpha, Beta, Gamma);
};

void totalSum ( Alpha x, Beta y, Gamma z) {
  int sum = x.a + y.b + z.c;
  cout << "Sum of all values: " << sum << endl;
}
  int main () {
  Alpha A1(10);
  Beta B1(20);
  Gamma C1(30);
  totalSum ( A1, B1, C1);
return 0;
}
```

12) Create a class Point with private members x & y. Write a friend function that calculates & returns the distance between two point objects.

```
#include <iostream>
#include <cmath>
using namespace std;
Class Point {
private:
  int x, y;
public:
  Point (int xVal, int yVal) {
     x = x.Val;
     y = y.Val;
  }
  friend double calculateDistance (Point, Point);
};
double calculateDistance (Point p1, Point p2){
  int dx = p2.x - p1.x;
  int dy = p2.y - p1.y;
   return sqrt( dx * dx + dy * dy);
}
  int main (){
  Point p1 (3,4);
  Point p2 (7,1);
  double distance = calculateDistance (p1, p2);
  Cout << " Distance between points: " << distance <<
                                          endl;
  return 0;
}
```

Page No.:
Date:

15) Create two classes : BankAccount & Audit. with private data members. Write a friend function in Audit that accesses & prints balance.

```
#include <iostream>
using namespace std;
class Audit;
class BankAccount {
private:
int balance;
public:
BankAccount(int b) {
    balance = b;
}
friend void auditBalance ( BankAccount, Audit);
};
class Audit{
~~private~~:
~~int~~ public:
void startAudit (BankAccount acc){
auditBalance (acc, *this);
}
friend void auditBalance (BankAccount, Audit);
};
(~~void accountBal~~)
void auditBalance (BankAccount , Audit) {
cout<<"Auditing Account = " << endl;
cout<<" balance: ₹ " << acc.balance <<endl;
}
```

```
int main() {
    BankAccount myAcc(500000);
    Audit auditor;
    auditor.startAudit(myAcc);
    return 0;
}
```

Q1
12/8

Page No.:
Date:

Experiment 5

8] WAP to find the sum of numbers from 1 to n. using (a) default constructor (b) Parameterized constructor where value of n will be passed to the constructor.

(a)

```
#include <iostream>
using namespace std;
class sum
{
   int n;
  int sum;
  public:
   sum() {
      sum=0;
     cout<<" Enter value of n:" <<endl;
     cin >>n;
     for(i=1; i<=n; i++){
          sum+=i;
      }
   }
   void display(){
      cout<<" Sum:"<<sum<<endl;
   }
};
  int main(){
     sum S;
     S.display();
    return 0;
  }
```

(b) Parameterized

```
#include <iostream>
using namespace std;

class sum
{
  int n, sum;
  public:
    Sum( int value){
        n = value;
        sum = 0;
       [cout << " Enter value of n :" << endl;
        cin >> n; ]
       for(int i=1; i<=n; i++) {
         sum += i;
       }
    }
    void display() {
    cout << " Sum is" << sum << endl;
    }
};
  int main(){
   int num;
   cout << " Enter n:";
   cin >> num;
   Sum s(num);
   s.display();
   return 0;
  }
```

(c) Copy

```
#include <iostream>
using namespace std;
class Sum
{
int n;
int s=0;
public:
Sum()
{
  n=4;
}
Sum(Sum &obj)
{
  n=obj.n;
  int i;
for(i=0; i<=n; i++)
{
  s=s+i;
}
cout<<"Sum: "<<s;
}
};
int main(){
Sum s1;
Sum s2(s1);
return 0;
}
```

Q] WAP to declare class 'student' having data members as name and percentage. Write a constructor to initialize these data members. Accept & display data for 1 student

(a) Default Constructor

```
#include <iostream>
using namespace std;
#include <string>

class Student
{
  string name;
  float percentage;
  public:
  Student() {
    name = " ";
    percentage = 0.0;
  }
  void
    cout<<" Enter student name: ";
    cin >> name;
    cout<<" Enter percentage: ";
    cin >> percentage;
  }

  void display()
  {
```

```
    name = 'Spruha';
    percentag = 90;
  }
  void display (){
  cout<<" Name of Student : " << name << "
  Percentage : " <<inpercentage <<endl;
  }
};

int main (){
  Student St;
  St.display ();
  return 0;
}
```

(b) Parameterized Constructor

```
#include <iostream>
using namespace std;

class student
{
    String name;
    float percentage;
  public:
  student (string n, float p){
      name = n;
    percentage = p;
  }
```

```
void display(){
cout << " Name of student: " << name <<
    "\nPercentage: " << percentage << endl;
}
};
int main(){
Student s1 ("Sparsha", 97.6);
 s1.display();
return 0;
}
```

(c)

```
#include <iostream>
#include <string>
using namespace std;
class Student
{
private:
string name;
float p;
public:
Student(string n, float per)
 {
  name = n;
  percentage = per;
 }
Student(const Student &s)        // Copy
{
```

```
    name = s.name;
    percentage = s.percentage;
    }
    void display(){
    cout<<"Name:"<<name;
    cout<<"Percentage:"<<p<<"%"<<endl;
    }
};

int main(){
Student s1("HSpruha", 98);
Student s2 = s1;
s2.display();
return 0;
}
```

Q3] Use default constructor

```
#include<iostream>
using namespace std;
class College
{
int roll_no;
string name;
string course;
public:
College(int n, string n, string c = "Computer Engg.")
{
  roll_no = n;
  name = n;
  course = c;
}
```

```
void display(){
cout<< "Roll no:" << roll_no << "Name:" << nam
"Course:" << course << endl;
}
};

int main() {
College s1(1, "Sprusha");
College s2(2, "Aditya", "IT");
s1.display();
s2.display();
return 0;
}
```

Q4] Code to demonstrate constructor overloading.

```
#include <iostream>
using namespace std;
class student
{
 int roll_no;
string name;
public:
student()
{
 roll_no = 7;
 name = "Anushka";
}
```

```
Student (int n)                    // para
{
  roll_no = n;
  name = "Sahil";
}
Student (int n, String n)          // para.
{
  roll_no = n;
  name = n;
}
void display ()
{
cout << "Roll-no:" << roll_no << "Name:" << name
 << endl;
}
};
 int main (){
 Student s1;
Student s2(21);
Student s3(25, "Spurtha");
s1.display ();
s2.display ();
s3.display ();
return 0;
}
```

16/9/25

Page No.:
Date:

Experiment 6

q1] Create a base class called Person with attributes name and age. Derive a class Student from Person that adds an attribute rollNumber. Write functions to display all details of the student.

```
#include <iostream>
using namespace std;
class Person
{
  protected:
  string name;
  int age;
  public:
  void accept (){
  cout<<"Enter name:";
   cin >> name;
   cout <<"Enter age:";
    cin >> age;
  }

  void display (){
  cout<<"Name:" << name << endl;
  cout<<"Age:" << age << endl;
  }
};

class Student : public Person {
private:
  int rollNumber;
```

```
public:
void acceptStudentDetails(){
 accept();
  cout<<"Enter roll number:";
  cin>>rollNumber;
}

void displayStudentDetails(){
   display();
  cout<<" Roll Number:"<<rollNumber<<endl;
  }
};

int main(){
 Student s;
s.acceptStudentDetails();
s.displayStudentDetails();
return 0;
}
```

Page No.:
Date:

Q2] Create 2 base class Academic & Sports for mul
Inheritance.
- Academic class - marks of student.
- Sports class - score.
- Derived class - Result inherited from both classes &
total score

```
#include <iostream>
using namespace std;

Class Academic {
 public:
  int marks;
  void getmarks() {
     cout << " Enter academic marks: ";
     cin >> marks;
   }
};

Class Sports {
 public:
    int score;
    void getscore() {
    cout << " Enter sport score: " ;
     cin >> score;
    }
};

Class Results : public Academic, public Sports {
public:
    void display() {
```

```
    int total = marks + score;
    cout << "\n Academic marks: " << marks;
    cout << "\n Sports score: " << score;
    cout << "\n Total Score: " << total << endl;
  }
};

int main(){
  Result R;
  R.getMarks();
  R.getScore();
  R.display();
  return 0;
}
```

Q3] Create a class vehicle with attributes brand & model
Derive class 1 - attributes type.
Derive class 2 - car battery capacity.
Display all functions.

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
public:
    string type brand, model;
};

class Car : public Vehicle {
public:
    string type;
};

class ElectricCar : public Car {
public:
    int batteryCapacity;
    void input() {
        cout << "Enter brand: ";
        cin >> brand;
        cout << "Enter model: ";
        cin >> model;
        cout << "Enter type: ";
        cin >> model; type;
```

```
    cout<<" Enter battery capacity (kWh): ";
    cin >> batteryCapacity;
}

void display() {
cout<<"\n -- Car details --";
cout<<" Brand: " << brand << endl;
cout<<" Model: " << model << endl;
cout<<" Type: " << type << endl;
cout<<" Battery capacity: " << batterCapacity <<
                                    endl;
}
};

int main(){
 ElectricCar e;
  e.input();
  e.display();
  return 0;
}
```

Page No.:
Date:

Q4] Hierarchial Inheritance.

```
#include <iostream>
using namespace std;
class Employee {
public:
    int empID;
    string name;
    void setData(int id, string n) {
    empID = id;
    name = n;
    }
    void display() {
    cout << "Employee ID:" << empID << endl;
    cout << "Name:" << name << endl;
    }
};

class Manager : public Employee {
public:
    string department;
    void setDepartment(string dept) {
    department = dept;
    }
    void display() {
    Employee :: display()
    cout << "Department:" << department << endl;
    }
};
```

```
class Developer : public Employee
{
public:
string programminglang;
void setProgrammingLang (string lang) {
programminglang = lang;
}
void display() {
Employee::display()
cout<<"Programming language:"<<programming-
lang<<endl;
}
};

int main() {
Manager m;
m.setData(101,"Sruha");
m.setDepartment("CEO");
Developer d;
d.setData(102,"Adwitee");
d.setProgrammiglang("Python");
cout<<"Manager Details:"<<endl;
m.display();
cout<<"\n Developer Details:"<<endl;
d.display();
return 0;
}
```

Q5] Hybrid Inheritance

```
#include <iostream>
using namespace std;
class Person {
  public:
    string name;
    int age;
    void setPerson (string n, int a) {
      name = n;
      age = a;
    }
    void displayPerson () {
    cout << " Name: " << name << endl;
    cout << " Age " << age << endl;
    }
};

class Student : public Person
  {
  public:
    int studentID;
    void setStudent (int ID) {
      studentID = ID;
    }
    void displayStudent () {
    cout << " Student ID: " << studentID << endl;
    }
};
```

Page No.:
Date:

```
class Sports
{
public:
  int sportsScore;
  void setScore (int score) {
    SportsScore = score;
  }
  void displayScore ()
  {
  cout << " Sports Score: " << sportsScore << endl;
  }
};

class Result: public Student, public Sports
{
public:
  int marks;
  void setMarks (int m) {
  marks = m;
  }
  void se
  int calculateTotal () {
  return marks + sportsScore;
  }
  void displayResult() {
    display Person();
    display Student();
    display Sports();
  cout << " Marks: " << marks << endl;
  cout << " Total (marks + Sports score): " <<
    Calculate Total << endl;
```

```
    }
};
int main(){
Result R;
R.setPerson("Bob", 35);
R.setStudent("R01", 15, 102);
R.SetSportScore(45);
R.setmarks(90);
R.displayResult();
return 0;
}
```

17/10

Page No.:
Date:

Experiment 7

1] WAP using function overloading to calculate the area of laboratory (rectangle) and area of class-room (square).

```
#include <iostream>
using namespace std;

class Area
{
public:
float calculate (float length, float breadth){
    return length * breadth;
}

float calculate (float side) {
    return side * side;
}
};

int main () {
  Area a;
  float length, breadth, side;

cout << " Enter length & breadth for laboratory: ");
cin >> length >> breadth;
cout << " Area of laboratory (rectangle) : "); <<
        a.calculate (length, breadth) << endl;
cout << " Area of classroom (square):") << a.calculate
        (side) << endl;
return 0;
}
```

Q2] WAP using function overloading to calculate sum of 5 float values & 10 integer values.

```
#include <iostream>
using namespace std;

class SumCalculator {
public:

  float sum(float a, float b, float c, float d, float e) {
    return a+b+c+d+e;
  }

  int sum(int a, int b, int c, int d, int e, int f, int g, int h, int i,
          int j) {
    return a+b+c+d+e+f+g+h+i+j;
  }
};

int main() {
  SumCalculator s;
  float fsum = s.sum(1.1f, 2.2f, 3.3f, 4.4f, 5.5f);
  int isum = s.sum(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
  cout << "Sum of 5 float nos.:" << fsum << endl;
  cout << "Sum of 10 integer nos.:" << isum << endl;
  return 0;
}
```

Page No.:
Date:

8] WAP to implement Unary(-) operator when used with the object so that numeric data member of the class is negated.

```
#include <iostream>
using namespace std;
class number
{
  int a;
  public:
  void accept() {
  cout<< "a =" << endl;
   cin >> a;
  }
  void display (){
   cout << " a =" << a << endl;
  }
   void operator -()
    {
      a = -a;
    }
};

int main () {
number n1;
n1.accept() ;
-n1;
n1. display () ;
return 0;
}
```

Page No.:
Date:

Q4] WAP to implement unary ++ operator (for pre & post increment) when used with the object so that the numeric data member of the class is incremented.

```
#include <iostream>
using namespace std;

class Number{
  int num;
public:
  Number(int n=0){
    num = n;
  }

  Number operator ++(){
      ++num;
      return * this;
  }

  Number operator ++ (int){
      Number temp = *this;
      num++;
      return temp;
  }

  void display(){
    cout << "Number: " << num << endl;
  }
};
```

```
int main(){
  Number n1(5);
  cout << " Inbial value :" << endl;
  n1.display();
  ++n1;
  cout << " After pre-increment :" << endl;
  n1.display();
  n1++;
  cout << " After post-increment :" << endl;
  n1.display();
  return 0;
}
```

17/10

Experiment 8

S1] Concatinated x y z & p q r: using '+' operator.

```
#include <iostream>
#include <string>
using namespace std;

class AddString{
public:
    string str;

class String1 {
 private:
  string str ;
 public:
  void accept() {
  cout << " Enter a string: " ;
     cin >> str;
 }
  void display() {
   cout << " Concatinated string: " << str ;
 }

 void operator + (String1 & s1)
  {
    strcat (str, s1.str );
  }
};
int main() {
```

```
String s1,s2;
 s1.accept();
 s2.accept();
 s1+s2;
 s1.display();
 return 0;
}
```

Q2]

```
#include <iostream>
#include <string>
using namespace std;

class Ilogin {
protected:
  string name, password;
public:
  virtual void accept() {
  cout << " Enter name: ";
   cin >> name;
  cout << " Enter password: ";
    cin >> password;
  }

  virtual void display(){
  cout << " Name: " << name << " Password: "
   password << endl;
  }
};

class Emaillogin : public Ilogin {
  string email;
public:
    void accept() {
    cout << " Enter email login!!";
    cin >> name;
    cout << " Enter password: ";
    cin >> password;
```

```
    cout << " Enter email ID : ";
    cin >> email ;
  }
  void display () {
  cout << " Name : " << name << " Password : " << password <<
  "Email ID " << email << endl;
  }
};

class membership : public Ilogin {
  int memberID;
  public:
  void accept() {
  cout << " Enter membership login : " << ;
    cin >> name;
    cout << " Enter password : ";
    cin >> password;
    cout << " Enter membership ID";
    cin >> memberID;
  }
  void display () {
cout << " Membership login : " << name << " Password "
  << password << " login ID : " << endl;
  }
};
```

Q

17/10

Page No.:
Date:

Experiment - 9

* Creating / opening file using open() function

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
 fstream new_file;
 new_file.open ("new_file.txt", ios::out);
  if (!new_file){
   cout<<"File creation failed"<<endl;
  }else{
   cout<<"New file created"<<endl;
    new_file.close();
  }
return 0;
}
```

Page No.:
Date:

* Copy content from one file to another:

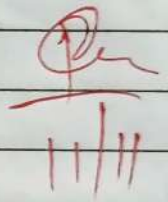
```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
  ofstream file("file1.txt", ios::app);
  if (!file){
    cout<<" Error opening file1.txt" << endl;
    return 1;
  }

  file1 << " Welcome to MIT " << endl;
  file1.close();

ifstream file1_read ("file1.txt");
ofstream file2 ("file2.txt");

  if (!file1_read) {
  cout << " Error opening file1.txt for reading << endl;

  return 1;
  }

  if (!file2) {
  cout<<" Error opening file2.txt for writing"<< endl;

  return 1;
  }
```

```
    file2.put(ch);
}
cout<<" Content copied successfully from
 file1.txt to file2.txt" << endl;

 file1_read.close();
 file2.close();
 return 0;
}
```

* count number of digits & spaces

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
ifstream inputFile ("input.txt");
 if (!inputFile){
  cout <<" Cannot open input.txt" <<
                                  endl;
  return 1;
}

 char ch;
 in  spacecount = 0;
     digit count = 0;
```

```
while (inputFile.get(ch)){
 if (ch == ' ') or isspace(ch)
     spacecount ++;
 else if (isdigit(ch))
        digit count ++;
}

    inputfile.close();

cout << " Total spaces: " << spacecount;
cout << " Total digits: " << digitcount;

return 0;
}
```



Page No.:
Date:

Experiment - 10

Find sum of array using function template. Integer, float, Double.

```
#include <iostream>
using namespace std;

template <class T>
T findSum (T arr[], int size){
   T sum = 0;
  for (int i=0; i<size; i++){
     sum += arr[i];
  }
  return sum;
}

int main(){
  const int size = 10;
// integer int intArr[size] = {1,2,3,4,5,6,7,8,9,10};

cout<<" Sum of int array :" << findSum (Arr, size)
<< endl;

return 0;
}

// float

float floatArr[size] = {1.1f, 2.2f, 3.3f, 4.4f, 5.5f, 6.6f
                        7.7f, 8.8f, 9.9f, 10.1f};
cout<<" Sum of float array :" << findSum (
```

floatArr, size) << endl;

double doubleArr[size] = {1.11, 2.22, 3.33, 4.44, 5.55,
6.66, 7.77, 8.88, 9.99, 10.10};
cout << "Sum of double array: " << findSum(doubleArr, size) << endl;

Page No.:
Date:

8] Concatination of string.

```
#include <iostream>
#include <cstring>
using namespace std;

template <class T>
T square (T x) {
    return x * x;
}

template <>
char * square <char *> (char * str) {
    static char result [200];
    strcpy (result, str);
    strcat (result, str);
    return result;
}

int main() {
    char str[100] = "Hello World";
    cout << "Square of string :" << square (str) << endl;

    return 0;
}
```

Page No.:
Date:

1 Calculator to perform 10 functions.

```
#include <iostream>
using namespace std;

template <class T> class calculator
{
public:
T num1 = 0;
T num2 = 0;

void accept(){
cout << " Enter num1 and num2: "
                                << endl;
} cin >> num1 >> num2;
}
void add(){
cout << " Sum: " << num1 + num2;
}

void subtract(){
cout << " Subtract: " << num1 - num2;
}

void mul(){
cout << " Multiply: " << num1 * num2;
}

void div(){
cout << " Division: " << num1 / num2;
}
```

```
void sine(){
cout << " Sine : " << sin(num1) << sin(num2);
}

void cosine(){
cout << " cosine: " << cos(num1) << cos(num2);
}

void Tan(){
cout << " Tan : " << tan(num1) << tan(num2);
}

void power(){
cout << " power: " << pow(num1, num2) << endl;
}

void squareroot(){
cout << " Square Root : " << sqrt(num1) <<
                              sqrt(num2);
}

void display(){
  cout << " \n-- Calculator menu -- " << endl;
  cout << " 1. Addition " << endl;
          2. Subtraction
          3. Multiplication
          4. Division
```

5. Modulus
6. Sine Power
7. Cosine
8. Tangent
9. ~~Power~~ Sine
10. Square root

}
};

```
int main(){
    int choice;
    Calculator<int> calc;
    cerl
    do{
      calc.display();
      cout<<" In Enter your choice: ";
      cout cin >> choice;

      if (choice >= 1 && choice <= 10)
        calc.accept();
      else if (choice >= 7 && choice <= 10){
        cout<<" Enter

      }

      cout << " Result " << endl;

    switch( choice){
      case 1 : calc.add() << endl;
        break;
```

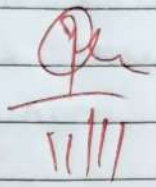
```
        calc.subtract();
        break;
    case 3: calc.mul();
        break;
    case 4: calc.div();

    case 5: calc.mod();

    case 6: calc.power();

    case 7: calc.cosine();

    case 8: calc.tan();

    case 9: calc.sine();

    case 10: calc.squareroot();
        break;
    default: cout<<" Invalid! "<<endl;
        break;
    }
    while( choice !>10 && choice != 0);

    return 0;
}
```

WAP to implement push & pop methods from stack using class template.

```
#include <iostream>
using namespace std;

template <class T> class stack{
    T arr[20];
    int top;
public:
    stack() {
    top = -1;
    }

void push (T value) {
    if (top >= 9)
        cout << "Stack overflow!" << endl;
    else {
        top++;
        arr[top] = value;
    cout << "value pushed into stack" << endl;
    }
}

void pop (T value){
    if (top == -1)
    cout << "Stack Underflow" << endl;
    else {
        cout << 'arr[top] popped from stack"
                                    << endl;
        top--;
    }
}
```

```
void display() {
  if (top == -1)
  cout << " Stack is empty " << endl;
  else {
  cout << " Stack elements: " ;
  for (int i = top; i >= 0 ; i--)
    cout << " arr[i] << " " << endl;
    }
  }
};

int main() {
  stack <int> s;
  int choice, value;

  do {
  cout << " \n 1. Push \n 2. Pop \n 3. Display
  \n 4. Exit \n Enter your choice: " ;
  cin >> choice;

  switch (choice) {
    case 1: cin >> value;
    s. push (value);
    break;
    case 2 :
      s. pop;
    break;
    case 3:
      s. display();
      break;
    case 4:
    cout << Exit << endl;
      break;
```

```
}
} while (choice != 0);

return 0;
}
```



Page No.:
Date:

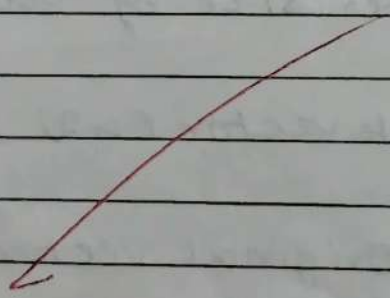
Experiment - 11

8] Create a vector.
1) Modify
2) Multiply by scalar.
3) Display.

```
#include <iostream>
#include <vector>
using namespace std;

template <class T> class Vector{
    vector <T> v;
public:
    void createVector (int n){
    cout << " Enter " << n << " elements : \n";
      v.resize(n);
      for (int i=0; i<n; i++)
          cin >> v[i];
    }

    void modify (int index, T value){
     if (index >=0 && index < v.size())
        v[index] = value;
     else
        cout << " Invalid ! \n";
    }

    void multiplybyscalar (T scalar){
     for (int i=0; i < v.size(); i++)
        v[i] *= scalar;
    }
```

```
void display() {
    cout << "(";
    for (int i=0; i< v.size(); i++){
        cout << v[i];
        if (i != v.size()-1)
            cout << ",";
    }
    cout << ")\n";
  }
};

int main(){
Vector <int > vec;
int n, index, newvalue, scalar;

cout << "Enter size of vector: ";
cin >> n;
vec.createvector(n);

cout << "Original vector: ";
  vec.display();

cout << "\n Enter index & newvalue to
    modify :";
cin >> index >> newvalue;
vec.modify(index, newvalue);

cout << "After modification:";
  vec.display();
```

```
cout << "\n Enter scalar to multiply: ";
cin >> scalar;
vec.multiply(scalar);

cout << "\n After multiplication: ";
vec.display();
return 0;
}
```



Date:

9] Using iterator

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
  vector<int> v = {2,4,6,8,10,12,14,16,18,20};

  cout << " Initial Vector : " << endl;
   for (vector<int>::iterator it = v.begin();
   it != v.end(); it++) {
    cout << *it << " " << endl;
   }

   for (vector<int>::iterator it = v.begin();
it != v.end(); it++) {
       *it = (*it) * 10;
   }

cout << " New Vector : " << endl;
 for (vector<int>::iterator it = v.begin();
it != v.end(); ++it) {
   cout << *it << " " << endl;
}

return 0;
}
```

Page No.:
Date:

Experiment - 12

a) WAP using STL to implement stack

```
#include <iostream>
#include <stack>
using namespace std;

int main(){
  stack<int> s;

cout << " Stack ";

s.push(10);
s.push(20);
s.push(30);

cout << " " << s.pop() << endl;
cout << " Display elements: ";
stack<int> temp = s;
while (!temp.empty()){
    cout << temp.top() << " ";
    temp.pop();
}
cout << endl;

s.top(5); cout << top element: << s.top();

return 0;

}
```

b) LAP using STL to implement Queue.

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
  queue<int> q;

q.push(10);
q.push(25);
q.push(45);

cout << "Queue elements:\n" << endl;
while queue<int> temp = q;
while (!temp.empty()) {
    cout << temp.front() << " ";
    temp.pop();
}
cout << endl;

return 0;
}
```

11/11